TermiCode AI — Simple Guide

How It Works, System Architecture & Workflow Profiles (Made Simple for Anyone)

1. The 10-Second Elevator Pitch (Explain to a Friend)

Imagine an AI assistant like ChatGPT or Claude, but instead of chatting in a browser, it lives directly inside your terminal. It can read your code files, diagnose failing tests, and write fixes. But unlike standard AI agents that can run wild and accidentally delete files, **TermiCode AI has a strict 'Safety Bouncer'** built around it:

- ■ **Zero-Trust Jail:** It mathematically cannot escape your project folder or touch sensitive files like .env or .ssh.
- ■ **Human-in-the-Loop:** It CANNOT write a single file or run a command without showing you a green/red diff and asking [y/N] permission.
- ■ **100% Local & Free:** It runs completely offline using local open-source models (Ollama / Qwen 2.5 Coder) with $0 API costs and zero data leaks.

2. The 4-Layer System Architecture

TermiCode AI is designed like a bank vault with 4 distinct protective layers:

Layer	Component	Plain English Explanation
1. CLI	Typer & Rich Console	The user interface. Formats beautiful tables, status boards, and interactive unified diff prompts.
2. Runtime	ReAct Agent Loop	The 'Brain'. Reads files, thinks, and proposes actions. Strictly capped at 10-25 steps so it never enters infinite loops.
3. Policy	Risk-Tier Engine	The 'Bouncer'. Classifies actions into **READ** (safe), **WRITE** (needs approval), and **EXECUTE** (needs approval).
4. Sandbox	Workspace Boundary	The 'Jail'. Canonicalizes paths, blocks ../ escapes, guards secrets, and runs subprocesses with shell=False.

3. The 6 Workflow Profiles (Role-Based AI)

Instead of a generic agent that tries to do everything, TermiCode AI uses **specialized workflow profiles**. Each profile has hard-coded tool restrictions and strict step budgets:

Workflow	Policy	Budget	Permitted Tools	What it Does
`understand`	**Read-Only**	10 steps	list, read, search	Explains architecture or answers questions. Zero chance of editing files.
`review`	**Read-Only**	10 steps	read, search, git diff	Inspects git diffs and critiques code for security, style, and logic bugs.

`generate`	Read/Write/Exec	25 steps	All tools (gated)	Implements a requested feature or module after explicit user approval.
`debug`	Read/Write/Exec	20 steps	All tools (gated)	Runs pytest, analyzes stack traces, and applies a minimal targeted patch.
`tests`	Read/Write/Exec	20 steps	All tools (gated)	Generates unit test suites and runs them to verify coverage.
`refactor`	Read/Write/Exec	15 steps	All tools (gated)	Restructures code for clean architecture while verifying tests pass.

4. Step-by-Step Task Lifecycle (A Real Example)

Here is the exact lifecycle of what happens when you type: termicode generate "Add a helper to format byte sizes"

Step 1: Workspace Check	TermiCode locks the current directory as the root. Any path containing ../, .env, or external drives is instantly forbidden.
Step 2: Model Inspection	The AI asks to call list_files(). Policy Engine sees Risk Level = **READ**. This is safe, so it executes automatically.
Step 3: Mutation Proposal	The AI crafts the code and asks to call write_file(path='src/helpers.py', content=...). Policy Engine sees Risk Level = **WRITE**. It immediately **HALTS** execution.
Step 4: Interactive Approval	A colorized diff preview is shown in the terminal: additions in green (+) and deletions in red (-). TermiCode asks: Approve? [y/N]:. If you press Enter or type 'n', it safely aborts.
Step 5: Execution & Memory	If you type 'y', the file is written. The full conversation, tool duration, and decisions are written to .termicode/runs/run_xxx.jsonl with all API keys automatically redacted.

5. The 3 Ways to Run TermiCode AI

TermiCode AI supports 3 distinct execution engines for maximum flexibility:

Mode	Command to Activate	Best For
1. Local Ollama (Local AI)	`$env:TERMICODE_PROVIDER="ollama"` `termicode status`	**Interviews & Enterprise:** Real local AI (Qwen 2.5 Coder) running on your PC. 100% free, private, no API key needed.
2. Fake Mode (Mock Testbed)	`$env:TERMICODE_PROVIDER="fake"` `termicode understand "..."`	**Instant Demo & Testing:** Zero setup, zero downloads. Tests approval prompts and diff previews with 0 tokens.
3. OpenAI Cloud (Cloud API)	`$env:OPENAI_API_KEY="sk-..."` `termicode generate "..."`	**Production Heavy Coding:** Powered by GPT-4o for massive multi-file feature synthesis.

6. Key Takeaways to Tell Your Friend or Interviewer

If someone asks: *'Isn't this just another ChatGPT wrapper?'*, here are the 4 points that prove it is serious systems engineering:

- **1. Defense-in-Depth Security:** It enforces real containment (path canonicalization via os.path.realpath, symlink blocking, sensitive credential protection for .env and .ssh).
- **2. Subprocess Containment:** Shell execution uses shell=False, strict command allowlisting (python, pytest, ruff, mypy, git), 30s timeouts, and 64 KB output buffer caps.
- **3. Deterministic Test Suite:** Achieves **285 test cases (284 passed, 1 skipped) in 10 seconds** in CI using the built-in FakeProvider testbed with 100% Mypy and Ruff compliance.
- **4. Complete Data Privacy:** Solves the #1 enterprise blocker for generative AI by keeping all code and inference strictly on localhost via Ollama.

Quick Command Summary:
• Check status: termicode status
• View workflows: termicode workflows
• Read-only check: termicode understand "explain project"
• Safe dry-run: termicode generate "new feature" --dry-run
• Run tests: pytest (285 automated tests)